

BOOKFORGE AI - COMPLETE AUTHOR-CONTROLLED SYSTEM

YOU CREATE. YOU CONTROL. WE ASSIST.

PHILOSOPHY: AUTHOR-FIRST DESIGN

BookForge is YOUR writing assistant, NOT an auto-generator.

- You write your story, your way
 - You maintain full creative control
 - AI assists only when YOU request it
 - Every word is YOUR decision
 - Your ideas, your characters, your plot
 - Available for ALL genres: Romance, Erotica, Sci-Fi, Fantasy, Thriller, Mystery, Horror, Literary Fiction
-

PART 6: REFINED APPLICATION - AUTHOR-CONTROLLED INTERFACE

// pages/WritingStudio.tsx - AUTHOR-CONTROLLED VERSION

```
import React, { useState, useEffect, useRef } from 'react';
import { useParams, useNavigate } from 'react-router-dom';
import axios from 'axios';
import {
  FiSave, FiDownload, FiUsers, FiMap, FiImage,
  FiMic, FiTrendingUp, FiArrowLeft, FiLightbulb,
  FiEdit3, FiBookOpen, FiZap, FiCheckCircle
} from 'react-icons/fi';
import ReactQuill from 'react-quill';
import 'react-quill/dist/quill.snow.css';

const WritingStudio: React.FC = () => {
  const { projectId } = useParams();
  const navigate = useNavigate();
  const [project, setProject] = useState<any>(null);
  const [currentChapter, setCurrentChapter] = useState<any>(null);
  const [content, setContent] = useState('');
  const [wordCount, setWordCount] = useState(0);
  const [showAssistPanel, setShowAssistPanel] = useState(false);
  const [autoSaveEnabled, setAutoSaveEnabled] = useState(true);
  const editorRef = useRef<any>(null);
```

```

// Calculate word count
useEffect(() => {
  const text = content.replace(/<[^>]*>/g, '');
  const words = text.trim().split(/\s+/).filter(w => w.length > 0);
  setWordCount(words.length);
}, [content]);

// Auto-save every 30 seconds
useEffect(() => {
  if (!autoSaveEnabled) return;

  const interval = setInterval(() => {
    saveContent(true);
  }, 30000);

  return () => clearInterval(interval);
}, [content, autoSaveEnabled]);

const saveContent = async (silent: boolean = false) => {
  if (!currentChapter || !project) return;

  try {
    const updatedChapter = {
      ...currentChapter,
      content,
      word_count: wordCount,
      updated_at: new Date().toISOString()
    };

    await axios.put(`http://localhost:8000/api/project/${projectId}`, {
      chapters: [
        ...(project.chapters || []).filter((c: any) => c.id !== currentChapter.id),
        updatedChapter
      ],
      current_word_count: (project.chapters || []).
        .filter((c: any) => c.id !== currentChapter.id)
        .reduce((sum: number, c: any) => sum + c.word_count, 0) + wordCount
    });

    if (!silent) {
      alert(' Saved successfully!');
    }
  } catch (error) {
    console.error('Error saving:', error);
    if (!silent) {

```

```

        alert('Failed to save');
    }
}
};

return (
  <div className="writing-studio min-h-screen bg-gray-900 text-white flex flex-col">
    {/* Top Navigation Bar */}
    <div className="bg-gray-800 border-b border-gray-700 px-6 py-3">
      <div className="flex justify-between items-center">
        {/* Left: Project Info */}
        <div className="flex items-center gap-4">
          <button
            onClick={() => navigate('/') }
            className="p-2 hover:bg-gray-700 rounded-lg transition"
          >
            <FiArrowLeft size={20} />
          </button>
          <div>
            <h1 className="text-lg font-bold">{project?.title || 'Untitled'}</h1>
            <div className="text-xs text-gray-400 flex items-center gap-3">
              <span>{project?.genre}</span>
              <span>•</span>
              <span>{wordCount.toLocaleString()} / {project?.target_word_count?.toLocaleString()}</span>
              <span>•</span>
              <span>{Math.round((wordCount / (project?.target_word_count || 1)) * 100)}%</span>
            </div>
          </div>
        </div>
      </div>

    {/* Right: Tool Buttons */}
    <div className="flex items-center gap-2">
      <button
        onClick={() => navigate(`/characters/${projectId}`)}
        className="p-2 hover:bg-gray-700 rounded-lg transition"
        title="Manage Characters"
      >
        <FiUsers size={20} />
      </button>
      <button
        onClick={() => navigate(`/plot/${projectId}`)}
        className="p-2 hover:bg-gray-700 rounded-lg transition"
        title="Plot Outline"
      >
        <FiMap size={20} />
      </button>
    </div>
  </div>
);

```

```

<button
  onClick={() => navigate(`/cover/${projectId}`)}
  className="p-2 hover:bg-gray-700 rounded-lg transition"
  title="Design Cover"
>
  <FiImage size={20} />
</button>
<button
  onClick={() => navigate(`/audiobook/${projectId}`)}
  className="p-2 hover:bg-gray-700 rounded-lg transition"
  title="Create Audiobook"
>
  <FiMic size={20} />
</button>
<button
  onClick={() => navigate(`/marketing/${projectId}`)}
  className="p-2 hover:bg-gray-700 rounded-lg transition"
  title="Marketing Tools"
>
  <FiTrendingUp size={20} />
</button>

<div className="w-px h-6 bg-gray-700 mx-2" />

<button
  onClick={() => saveContent()}
  className="bg-purple-600 hover:bg-purple-700 px-4 py-2 rounded-lg flex items-c
>
  <FiSave size={18} />
  Save
</button>
</div>
</div>
</div>

{/* Main Content Area */}
<div className="flex-1 flex overflow-hidden">
  {/* Left Sidebar - Chapter Navigation */}
  <ChapterSidebar
    project={project}
    currentChapter={currentChapter}
    onSelectChapter={(chapter) => {
      setCurrentChapter(chapter);
      setContent(chapter.content || '');
    }}
    onNewChapter={() => {

```

```

    const newChapter = {
      id: Date.now().toString(),
      number: (project?.chapters?.length || 0) + 1,
      title: `Chapter ${project?.chapters?.length || 0} + 1`,
      content: '',
      word_count: 0
    };
    setCurrentChapter(newChapter);
    setContent('');
  }}
/>

{/* Center - Writing Editor */}
<div className="flex-1 flex flex-col">
  {/* Editor Toolbar */}
  <EditorToolbar
    showAssistPanel={showAssistPanel}
    onToggleAssist={() => setShowAssistPanel(!showAssistPanel)}
    onExportChapter={() => exportChapter()}
    wordCount={wordCount}
    autoSaveEnabled={autoSaveEnabled}
    onToggleAutoSave={() => setAutoSaveEnabled(!autoSaveEnabled)}
  />

  {/* Main Editor */}
  <div className="flex-1 overflow-y-auto bg-gray-800 p-8">
    <div className="max-w-4xl mx-auto bg-white rounded-lg shadow-2xl min-h-full">
      <div className="p-12">
        {/* Chapter Title */}
        {currentChapter && (
          <input
            type="text"
            value={currentChapter.title}
            onChange={(e) => setCurrentChapter({ ...currentChapter, title: e.target.value })}
            className="w-full text-3xl font-bold mb-8 outline-none border-b-2 border-gray-800"
            placeholder="Chapter Title"
          />
        )}

        {/* Rich Text Editor */}
        <ReactQuill
          ref={editorRef}
          theme="snow"
          value={content}
          onChange={setContent}
          placeholder="Start writing your story here... You have full creative control"
        />
      </div>
    </div>
  </div>

```

```

modules={
  toolbar: [
    [{ 'header': [1, 2, 3, false] }],
    ['bold', 'italic', 'underline', 'strike'],
    [{ 'list': 'ordered'}, { 'list': 'bullet' }],
    [{ 'indent': '-1'}, { 'indent': '+1' }],
    ['blockquote'],
    [{ 'align': [] }],
    ['link'],
    ['clean']
  ]
}
className="text-black"
style={
  minHeight: '700px',
  fontSize: '18px',
  lineHeight: '1.8',
  fontFamily: 'Georgia, serif'
}
/>
</div>
</div>
</div>
</div>

{/* Right Sidebar - AI Assistant Panel (Optional) */}
{showAssistPanel && (
  <AIAssistPanel
    project={project}
    currentContent={content}
    onInsertText={(text) => {
      // Insert at cursor position
      setContent(content + '\n\n' + text);
    }}
    onReplaceText={(text) => {
      setContent(text);
    }}
  />
)}
</div>
</div>
);
};

export default WritingStudio;

```

```
// components/ChapterSidebar.tsx - Chapter Management
```

```
import React from 'react';
import { FiPlus, FiEdit2, FiTrash2, FiChevronRight } from 'react-icons/fi';
```

```
interface ChapterSidebarProps {
  project: any;
  currentChapter: any;
  onSelectChapter: (chapter: any) => void;
  onNewChapter: () => void;
}
```

```
const ChapterSidebar: React.FC<ChapterSidebarProps> = ({
  project,
  currentChapter,
  onSelectChapter,
  onNewChapter
}) => {
  return (
    <div className="w-72 bg-gray-800 border-r border-gray-700 flex flex-col">
      { /* Header */ }
      <div className="p-4 border-b border-gray-700">
        <button
          onClick={onNewChapter}
          className="w-full bg-purple-600 hover:bg-purple-700 px-4 py-3 rounded-lg flex items-center"
        >
          <FiPlus size={20} />
          New Chapter
        </button>
      </div>

      { /* Chapter List */ }
      <div className="flex-1 overflow-y-auto p-4 space-y-2">
        {project?.chapters?.length === 0 ? (
          <div className="text-center text-gray-400 py-8">
            <FiEdit2 size={48} className="mx-auto mb-3 opacity-30" />
            <p>No chapters yet.</p>
            <p className="text-sm mt-1">Click "New Chapter" to begin!</p>
          </div>
        ) : (
          project?.chapters?.map((chapter: any) => (
            <button
              key={chapter.id}
              onClick={() => onSelectChapter(chapter)}
              className={`w-full text-left p-4 rounded-lg transition-all group ${
                currentChapter?.id === chapter.id

```

```

      ? 'bg-purple-600 shadow-lg'
      : 'bg-gray-700 hover:bg-gray-650'
    }`}
  >
    <div className="flex items-start justify-between mb-2">
      <div className="font-semibold text-sm">
        Chapter {chapter.number}
      </div>
      {currentChapter?.id === chapter.id && (
        <FiChevronRight size={16} />
      )}
    </div>
    <div className="text-sm mb-2 line-clamp-1">
      {chapter.title}
    </div>
    <div className="text-xs text-gray-400">
      {chapter.word_count?.toLocaleString() || 0} words
    </div>
  </button>
))
  })
</div>

{/* Footer Stats */}
<div className="p-4 border-t border-gray-700 bg-gray-850">
  <div className="text-xs text-gray-400 space-y-1">
    <div className="flex justify-between">
      <span>Total Chapters:</span>
      <span className="font-semibold">{project?.chapters?.length || 0}</span>
    </div>
    <div className="flex justify-between">
      <span>Total Words:</span>
      <span className="font-semibold">
        {project?.current_word_count?.toLocaleString() || 0}
      </span>
    </div>
  </div>
</div>
);
};

export default ChapterSidebar;

// components/EditorToolbar.tsx - Writing Tools

```



```

import React from 'react';
import { FiLightbulb, FiDownload, FiZap, FiClock } from 'react-icons/fi';

interface EditorToolbarProps {
  showAssistPanel: boolean;
  onToggleAssist: () => void;
  onExportChapter: () => void;
  wordCount: number;
  autoSaveEnabled: boolean;
  onToggleAutoSave: () => void;
}

const EditorToolbar: React.FC<EditorToolbarProps> = ({
  showAssistPanel,
  onToggleAssist,
  onExportChapter,
  wordCount,
  autoSaveEnabled,
  onToggleAutoSave
}) => {
  const readingTime = Math.ceil(wordCount / 250); // 250 words per minute
  const pageCount = Math.ceil(wordCount / 250); // 250 words per page

  return (
    <div className="bg-gray-800 border-b border-gray-700 px-6 py-3">
      <div className="flex items-center justify-between">
        /* Left: Writing Tools */
        <div className="flex items-center gap-2">
          <button
            onClick={onToggleAssist}
            className={`px-4 py-2 rounded-lg flex items-center gap-2 transition ${
              showAssistPanel
                ? 'bg-gradient-to-r from-purple-600 to-pink-600'
                : 'bg-gray-700 hover:bg-gray-600'
            }`}
          >
            <FiLightbulb size={18} />
            <span className="font-semibold">AI Assist</span>
          </button>

          <div className="h-6 w-px bg-gray-700 mx-2" />

          <button
            onClick={onExportChapter}
            className="px-4 py-2 rounded-lg bg-gray-700 hover:bg-gray-600 flex items-center"
          >

```

```

        <FiDownload size={18} />
        <span>Export</span>
      </button>
    </div>

    {/* Right: Stats */}
    <div className="flex items-center gap-6 text-sm">
      <div className="flex items-center gap-2">
        <FiClock size={16} className="text-gray-400" />
        <span className="text-gray-400">Auto-save:</span>
        <button
          onClick={onToggleAutoSave}
          className={`px-3 py-1 rounded text-xs font-semibold ${
            autoSaveEnabled
              ? 'bg-green-600 hover:bg-green-700'
              : 'bg-gray-600 hover:bg-gray-500'
          }`}
        >
          {autoSaveEnabled ? 'ON' : 'OFF'}
        </button>
      </div>

      <div className="text-gray-400">
        <span className="font-semibold text-white">{wordCount.toLocaleString()}</span> w
      </div>

      <div className="text-gray-400">
        ~<span className="font-semibold text-white">{pageCount}</span> pages
      </div>

      <div className="text-gray-400">
        ~<span className="font-semibold text-white">{readingTime}</span> min read
      </div>
    </div>
  </div>
);
};

export default EditorToolbar;

// components/AIAssistPanel.tsx - OPTIONAL AI ASSISTANCE

import React, { useState } from 'react';
import axios from 'axios';
import {

```

```

    FiZap, FiRefreshCw, FiType, FiUsers,
    FiMessageSquare, FiBook, FiAlertCircle
  } from 'react-icons/fi';

  interface AIAssistPanelProps {
    project: any;
    currentContent: string;
    onInsertText: (text: string) => void;
    onReplaceText: (text: string) => void;
  }

  const AIAssistPanel: React.FC<AIAssistPanelProps> = ({
    project,
    currentContent,
    onInsertText,
    onReplaceText
  }) => {
    const [activeTab, setActiveTab] = useState<'generate' | 'ideas' | 'edit'>('generate');
    const [prompt, setPrompt] = useState('');
    const [generating, setGenerating] = useState(false);
    const [result, setResult] = useState('');

    const assistOptions = [
      {
        id: 'continue',
        icon: <FiType />,
        title: 'Continue Writing',
        description: 'AI suggests what comes next',
        prompt: 'Continue the story naturally from where I left off'
      },
      {
        id: 'dialogue',
        icon: <FiMessageSquare />,
        title: 'Add Dialogue',
        description: 'Create conversation between characters',
        prompt: 'Write dialogue between the characters in this scene'
      },
      {
        id: 'description',
        icon: <FiBook />,
        title: 'Add Description',
        description: 'Enhance scene descriptions',
        prompt: 'Add rich sensory description to this scene'
      },
      {
        id: 'character_action',

```

```

        icon: <FiUsers />,
        title: 'Character Action',
        description: 'Develop character movements/actions',
        prompt: 'Write what the character does next with vivid detail'
    }
];

const generateContent = async (customPrompt?: string) => {
    setGenerating(true);
    setResult('');

    try {
        const response = await axios.post('http://localhost:8000/api/generate', {
            project_id: project.id,
            prompt: customPrompt || prompt,
            genre: project.genre,
            heat_level: project.heat_level,
            characters: project.characters?.map((c: any) => c.name) || [],
            setting: 'Current scene',
            style: 'Match existing style',
            length: 300,
            context: currentContent.slice(-1000) // Last 1000 chars for context
        });

        setResult(response.data.content);
    } catch (error) {
        console.error('Error generating:', error);
        setResult('Error: Failed to generate content. Please try again.');
```

```

    } finally {
        setGenerating(false);
    }
};

return (
    <div className="w-96 bg-gray-800 border-l border-gray-700 flex flex-col">
        <div className="p-4 border-b border-gray-700">
            <h3 className="font-bold text-lg flex items-center gap-2">
                <FiZap className="text-purple-400" />
                AI Writing Assistant
            </h3>
            <p className="text-xs text-gray-400 mt-1">
                Optional help when you need inspiration
            </p>
        </div>
    </div>

```

```

{/* Tabs */}
<div className="flex border-b border-gray-700">
  <button
    onClick={() => setActiveTab('generate')}
    className={`flex-1 py-3 text-sm font-semibold transition ${
      activeTab === 'generate'
        ? 'bg-gray-700 text-purple-400 border-b-2 border-purple-400'
        : 'text-gray-400 hover:text-white'
    }`}
  >
    Generate
  </button>
  <button
    onClick={() => setActiveTab('ideas')}
    className={`flex-1 py-3 text-sm font-semibold transition ${
      activeTab === 'ideas'
        ? 'bg-gray-700 text-purple-400 border-b-2 border-purple-400'
        : 'text-gray-400 hover:text-white'
    }`}
  >
    Ideas
  </button>
  <button
    onClick={() => setActiveTab('edit')}
    className={`flex-1 py-3 text-sm font-semibold transition ${
      activeTab === 'edit'
        ? 'bg-gray-700 text-purple-400 border-b-2 border-purple-400'
        : 'text-gray-400 hover:text-white'
    }`}
  >
    Edit
  </button>
</div>

{/* Content Area */}
<div className="flex-1 overflow-y-auto p-4">
  {activeTab === 'generate' && (
    <div className="space-y-4">
      {/* Quick Actions */}
      <div>
        <h4 className="text-sm font-semibold mb-3 text-gray-300">Quick Actions</h4>
        <div className="space-y-2">
          {assistOptions.map((option) => (
            <button
              key={option.id}
              onClick={() => generateContent(option.prompt)}

```

```

        disabled={generating}
        className="w-full text-left p-3 rounded-lg bg-gray-700 hover:bg-gray-650"
      >
        <div className="flex items-start gap-3">
          <div className="text-purple-400 mt-1">{option.icon}</div>
          <div className="flex-1">
            <div className="font-semibold text-sm">{option.title}</div>
            <div className="text-xs text-gray-400 mt-1">{option.description}</div>
          </div>
        </div>
      </button>
    )})
  </div>
</div>

{ /* Custom Prompt */ }
<div>
  <h4 className="text-sm font-semibold mb-3 text-gray-300">Custom Request</h4>
  <textarea
    value={prompt}
    onChange={(e) => setPrompt(e.target.value)}
    placeholder="Describe what you want to write next... (e.g., 'Write a steamy
    rows={4}
    className="w-full bg-gray-700 rounded-lg px-3 py-2 text-sm text-white placeholder-gray-500"
  />
  <button
    onClick={() => generateContent()}
    disabled={generating || !prompt.trim()}
    className="w-full mt-2 bg-gradient-to-r from-purple-600 to-pink-600 hover:from-purple-500 hover:to-pink-500"
  >
    {generating ? (
      <>
        <FiRefreshCw className="animate-spin" />
        Generating...
      </>
    ) : (
      <>
        <FiZap />
        Generate
      </>
    )}
  </button>
</div>

{ /* Result */ }
{result && (

```

```

    <div className="mt-4">
      <h4 className="text-sm font-semibold mb-2 text-gray-300">Generated Content</h4>
      <div className="bg-gray-700 rounded-lg p-4 text-sm leading-relaxed">
        {result}
      </div>
      <div className="flex gap-2 mt-3">
        <button
          onClick={() => onInsertText(result)}
          className="flex-1 bg-green-600 hover:bg-green-700 px-4 py-2 rounded-lg"
        >
          Insert Below
        </button>
        <button
          onClick={() => setResult('')}
          className="px-4 py-2 rounded-lg bg-gray-600 hover:bg-gray-500 text-sm"
        >
          Clear
        </button>
      </div>
    </div>
  )}

  { /* Disclaimer */ }
  <div className="bg-blue-900 bg-opacity-30 border border-blue-700 rounded-lg p-3">
    <FiAlertCircle className="inline mr-2" />
    <strong>Remember:</strong> AI is a tool, not a replacement. Review and edit all
  </div>
</div>
)}

{activeTab === 'ideas' && (
  <IdeasTab project={project} />
)}

{activeTab === 'edit' && (
  <EditTab currentContent={currentContent} onReplaceText={onReplaceText} />
)}
</div>
</div>
);
};

// Ideas Tab Component
const IdeasTab: React.FC<{ project: any }> = ({ project }) => {
  const [ideas, setIdeas] = useState<string[]>([]);
  const [generating, setGenerating] = useState(false);

```

```

const generateIdeas = async (type: string) => {
  setGenerating(true);
  try {
    const response = await axios.post('http://localhost:8000/api/ideas/generate', {
      project_id: project.id,
      type,
      genre: project.genre
    });
    setIdeas(response.data.ideas || []);
  } catch (error) {
    console.error('Error generating ideas:', error);
  } finally {
    setGenerating(false);
  }
};

return (
  <div className="space-y-4">
    <h4 className="text-sm font-semibold text-gray-300">Need Inspiration?</h4>

    <div className="grid grid-cols-2 gap-2">
      <button
        onClick={() => generateIdeas('plot_twist')}
        disabled={generating}
        className="p-3 bg-gray-700 hover:bg-gray-650 rounded-lg text-sm font-semibold tran
      >
        Plot Twists
      </button>
      <button
        onClick={() => generateIdeas('dialogue_starter')}
        disabled={generating}
        className="p-3 bg-gray-700 hover:bg-gray-650 rounded-lg text-sm font-semibold tran
      >
        Dialogue Ideas
      </button>
      <button
        onClick={() => generateIdeas('scene_idea')}
        disabled={generating}
        className="p-3 bg-gray-700 hover:bg-gray-650 rounded-lg text-sm font-semibold tran
      >
        Scene Ideas
      </button>
      <button
        onClick={() => generateIdeas('character_moment')}
        disabled={generating}

```



```

        className="p-3 bg-gray-700 hover:bg-gray-650 rounded-lg text-sm font-semibold tran
    >
        Character Moments
    </button>
</div>

{ideas.length > 0 && (
    <div className="mt-4">
        <h5 className="text-xs font-semibold text-gray-400 mb-2">Suggestions:</h5>
        <ul className="space-y-2">
            {ideas.map((idea, index) => (
                <li key={index} className="bg-gray-700 rounded-lg p-3 text-sm">
                    {idea}
                </li>
            ))}
        </ul>
    </div>
)}
</div>
);
};

// Edit Tab Component
const EditTab: React.FC<{ currentContent: string; onReplaceText: (text: string) => void }> =
    currentContent,
    onReplaceText
}) => {
    const [instruction, setInstruction] = useState('');
    const [editing, setEditing] = useState(false);

    const rewriteContent = async (preset?: string) => {
        setEditing(true);
        try {
            const response = await axios.post('http://localhost:8000/api/rewrite', {
                text: currentContent,
                instruction: preset || instruction
            });
            onReplaceText(response.data.content);
        } catch (error) {
            console.error('Error rewriting:', error);
        } finally {
            setEditing(false);
        }
    };

    const presets = [

```

```

    { label: 'Make it more descriptive', value: 'Add rich sensory details and vivid descriptions' },
    { label: 'Make it more concise', value: 'Tighten the prose and remove unnecessary words' },
    { label: 'Increase tension', value: 'Add more tension and conflict to the scene' },
    { label: 'More emotional', value: 'Deepen the emotional impact and character feelings' },
    { label: 'More dialogue', value: 'Convert narration to dialogue where appropriate' },
    { label: 'Show don\'t tell', value: 'Convert telling statements to showing through action' }
  ];

```

```

return (
  <div className="space-y-4">
    <h4 className="text-sm font-semibold text-gray-300">Improve Your Text</h4>

    <div className="space-y-2">
      <h5 className="text-xs font-semibold text-gray-400">Quick Improvements:</h5>
      {presets.map((preset, index) => (
        <button
          key={index}
          onClick={() => rewriteContent(preset.value)}
          disabled={editing || !currentContent}
          className="w-full text-

```